

LLD Design Patterns

The memory map. One analogy per pattern, so you never forget which is which.

[@rathoreadiitya](#) - Instagram

v1.0 · Updated 2026-06

Inside this sheet

1. Start here: the student minimum - what a fresher round ACTUALLY tests
2. Why patterns feel impossible to remember - and the fix
3. OOP in one screen - the 4 pillars with desi analogies
4. SOLID in one screen - 5 principles, 5 analogies
5. THE PATTERN MEMORY MAP - all 12 at a glance (screenshot this)
6. Beyond the basics - how to read the deep dives
7. The 12 patterns - one analogy each (deep dive)
8. The confusion-killer - the 5 pairs people always mix up
9. Machine-coding rounds - which pattern shows up where
10. Interview triggers - if they say X, reach for Y
11. Common mistakes in LLD rounds - what loses points
12. The other 11 GoF patterns - recognize the words
13. Revision - the last 20 minutes before the round

If you have only 20 minutes -> jump to section 13 (Revision) and the memory map in section 5.

1. Start here: the student minimum

Honest truth first. A fresher LLD or machine-coding round does NOT test all 23 design patterns. Nobody is asking you to recite Visitor or Flyweight. What actually gets tested:

1. The 4 OOP pillars - encapsulation, abstraction, inheritance, polymorphism. You must explain each with a one-line example.
2. SOLID principles - at least name all five, and explain S, O, and D really well.
3. About 5 patterns you can actually apply in code: Singleton, Factory, Strategy, Observer, Decorator.
4. Designing ONE small system end to end. Parking Lot is the classic warm-up.

The one hands-on thing worth doing: code one machine-coding problem (parking lot) fully, even if it's ugly. Applying ONE pattern in real code beats memorizing ten definitions.

Time-pressed? Read the memory map (section 5), the confusion-killer (section 8), and the revision page (section 13). Skip the deep dives past the core 5 on your first pass.

2. Why patterns feel impossible to remember

DSA punishes wrong logic. Design patterns punish something sneakier: on paper, every pattern looks the same. They are all 'some classes calling other classes through an interface.' The names (Strategy, State, Bridge, Proxy) tell you nothing about what they do.

So students memorize 23 textbook definitions, mix up half of them under pressure, and blank when the interviewer asks the only question that matters: 'which pattern fits here?'

The fix is not more definitions. It is ONE physical picture per pattern. You will not remember 'Strategy encapsulates a family of algorithms.' You WILL remember Google Maps offering car, bike, or walk. The picture survives interview pressure. The definition does not.

That is the whole point of this sheet. Every pattern below gets one desi analogy. Learn the picture first, the formal definition second.

3. OOP in one screen - the 4 pillars

Every pattern is built on these four. If the analogy for a pattern feels shaky, it usually means one of these pillars is shaky. Fix these first.

Pillar	Desi analogy	What it means
Encapsulation	ATM machine - tum sirf buttons dabate ho, andar ka cash aur logic chhupa hai.	Bundle data + the methods that touch it inside one class. Outsiders cannot poke the internals directly.
Abstraction	Car ka steering - ghumao, gaadi mudti hai, andar ka mechanism janne ki zaroorat nahi.	Show only what the user needs. Hide the messy how behind a simple interface.
Inheritance	Bachcha parent se features paata hai, phir apna kuch add karta hai.	A child class reuses a parent's fields and methods, then extends or overrides some.
Polymorphism	'Run' word - insaan run karta hai, program run hota hai. Same word, alag matlab.	The same call (e.g. shape.area()) does the right thing for whichever concrete class is behind it.

New to OOP code? From here the pseudocode uses three words a lot: 'static' (one copy shared by the whole class), 'interface' (a contract that classes promise to follow), and 'override' (a subclass replacing a parent's method). If these are new, skim them in your Java or C++ notes before the deep dives.

4. SOLID in one screen

Five rules that keep classes clean. Interviewers love asking 'which SOLID principle does this violate?' Learn the smell each one fixes.

Principle	Desi analogy	The smell it fixes
S - Single Responsibility	Ek dukaan jo chai bhi beche, banaye aur cycle bhi repair kare - confusion.	A god class doing many unrelated jobs. One class = one reason to change.
O - Open / Closed	Plug-in socket - naya device lagao, wiring mat todo.	Editing tested code for every new case. Open to extend, closed to modify.
L - Liskov Substitution	Penguin is a Bird, but Bird.fly() crashes on Penguin - inheritance galat hai.	A subclass that breaks the parent's promise when used in its place.
I - Interface Segregation	Printer ko sirf print() chahiye, usse scan() aur fax() mat implement karwao.	Fat interfaces that force classes to implement methods they never use.
D - Dependency Inversion	Charger socket pe depend karta hai, kisi ek phone brand pe nahi.	Hard-wired 'new ConcreteClass' everywhere. Depend on interfaces, not concretes.

5. The pattern memory map - all 12 at a glance

This is the table to screenshot. The hook column is your one-line recall trigger. Learn the hook first, the formal definition second.

The 3 families of design patterns

Creational makes objects. Structural arranges them. Behavioral is how they talk and share work.

Creational How objects get made	Structural How objects are arranged	Behavioral How objects talk and share work
Singleton - ek hi Principal Factory - chai tapri Builder - Subway sandwich Abstract Factory Prototype	Adapter - universal plug Decorator - pizza layers Facade - waiter Proxy - club bouncer Bridge / Composite / Flyweight	Strategy - Maps route Observer - Insta follow State - traffic signal Command - order slip Template Method - Maggi recipe

One-line rule: 'new / build / clone' = Creational. 'wrap / fit / arrange' = Structural. 'notify / switch / undo / sequence' = Behavioral. Faded names are good-to-recognize, not must-code.

The 3 families. Creational = how objects are made. Structural = how they are arranged. Behavioral = how they talk.

#	Pattern	Type	One line to remember	What it does
1	Singleton	Creational	Ek hi banega, har baar wahi. (college ka ek Principal)	Makes sure a class has only ONE instance, and gives everyone one shared way to reach it.

#	Pattern	Type	One line to remember	What it does
2	Factory Method	Creational	Tum maango, kitchen banaye. (chai tapri, no 'new')	Moves object creation into a separate method so the caller asks for a product by name and never writes 'new ConcreteClass' itself.
3	Builder	Creational	Step by step, phir .build(). (Subway sandwich)	Builds a complex object step by step, so you avoid one giant constructor with a dozen parameters.
4	Adapter	Structural	Do na-milte plug ko jodo. (foreign plug, Indian socket)	Wraps one interface so it looks like the interface the client expects. Lets two incompatible classes work together.
5	Decorator	Structural	Base same, upar layers chadhao. (pizza topping)	Wraps an object to add new behavior at runtime, without changing the original class or any other object.
6	Facade	Structural	Ek simple window, peeche poora system. (waiter)	Gives one simple entry point that hides a complex subsystem of many classes behind it.
7	Proxy	Structural	Asli object se pehle ek gatekeeper. (club bouncer)	Puts a stand-in in front of the real object to control access, lazy-load, cache, or log, without changing the real object.
8	Strategy	Behavioral	Manzil same, raasta tum chuno. (Maps route)	Defines a family of interchangeable algorithms and lets you swap one for another at runtime, from outside.
9	Observer	Behavioral	Ek badla, sabko khabar. (Insta follow)	When one object (the subject) changes, all its registered watchers (observers) get notified automatically.
10	State	Behavioral	State badla, behavior badla. (traffic signal)	Lets an object change its behavior when its internal state changes, as if it switched class, instead of a giant if-else on a status field.
11	Command	Behavioral	Request ko parchi bana do. (order slip, undo bhi)	Turns a request into a standalone object, so you can queue it, log it, pass it around, and undo or redo it.
12	Template Method	Behavioral	Recipe fixed, ek step tumhara. (Maggi, sirf masala)	Defines the skeleton of an algorithm in a base class and lets subclasses override specific steps, without changing the overall structure.

Remember the 3 families in one line: CREATIONAL makes objects, STRUCTURAL arranges objects, BEHAVIORAL is about how objects talk and share work. If a pattern is about 'new / build / clone' it is Creational. About 'wrap / fit / arrange' it is Structural. About 'notify / switch / undo / sequence' it is Behavioral.

6. Beyond the basics (deeper than a fresher round needs)

The 12 deep dives below are the useful set for real interviews. For a fresher round, MASTER the core 5 - Singleton, Factory, Strategy, Observer, Decorator - well enough to code them. Treat the other 7 as recognize-the-shape: know the analogy and when it fits, but you will rarely be asked to implement them cold. Go deeper only for senior rounds.

7. The 12 patterns - one analogy each

1. Singleton (Creational)

Analogy: College ka ek hi Principal hota hai. Chahe din mein das baar office jao, wahi banda milta hai. Har visit pe naya principal nahi banta. Singleton bhi aisa hi: ek hi instance, baar-baar maango wahi milega.

What it does: Makes sure a class has only ONE instance, and gives everyone one shared way to reach it.

Spot it when you hear:

- 'There should be only one X in the whole system'
- 'A shared config / connection pool / logger / cache'
- 'A single global point of access'

Pseudocode

```
class Logger:
    _instance = null
    static getInstance():
        if _instance is null:
            _instance = new Logger()    # built once, lazily
        return _instance                # same object every call
```

Trap: With multiple threads, the lazy getInstance() above can build TWO instances if two threads hit it at the same time. **Fixes:** make getInstance() synchronized, use eager init (build it once when the class loads), or an enum singleton. Interviewers ask this almost every time. **And one mindset rule:** if your first instinct is Singleton, it is probably wrong. Use it only when the requirement truly demands one shared instance.

Real interview use: Logger, app config manager, DB connection pool, in-memory cache.

Don't confuse: Singleton controls HOW MANY exist (exactly one). Factory controls WHICH TYPE gets built. Also: Singleton is NOT just a global variable you read from anywhere. It is a controlled single entry point, so it stays testable and mockable.

2. Factory Method (Creational)

Analogy: Chai tapri pe tum bas bolte ho 'ek chai do'. Tum ye nahi batate kaise banegi. Kitchen decide karti hai kaunsi chai banegi, tumhare haath mein bas chai aati hai. Code mein bhi tum 'new Circle()' khud nahi likhte, factory se maang lete ho.

What it does: Moves object creation into a separate method so the caller asks for a product by name and never writes 'new ConcreteClass' itself.

Spot it when you hear:

- 'Create an object without knowing its exact class'
- 'Decide the type at runtime'
- 'A long if-else chain of new X(), new Y()...'

Pseudocode

```
class ShapeFactory:
    create(type):
        if type == 'circle': return new Circle()
        if type == 'square': return new Square()
# caller: shape = factory.create('circle') # no 'new Circle' here
```

Real interview use: Notification factory (email / SMS / push), shape creation, payment-gateway selection, parser creation.

Worth knowing: The if-else above is the teaching version. In production, factories usually use a registration map (type string -> creator) instead of a long if-else chain, so adding a new type never touches the factory code.

Don't confuse: Factory = ONE call, 'give me the right type'. Builder = MANY calls, 'assemble one complex object step by step'.

3. Builder (Creational)

Analogy: Subway sandwich socho. Pehle bread chuno, phir veggies, phir sauce, phir toast. Ek-

ek step pe object banta jaata hai, aur end mein '.build()' bolte hi poora sandwich milta hai. Constructor mein das cheezein ek saath thoosni nahi padti.

What it does: Builds a complex object step by step, so you avoid one giant constructor with a dozen parameters.

Spot it when you hear:

- 'An object with many optional fields'
- 'A telescoping constructor with too many params'
- 'Build an immutable object piece by piece'

Pseudocode

```
pizza = PizzaBuilder()  
    .size('large')  
    .cheese(true)  
    .addTopping('olives')  
    .build()           # final object assembled here
```

Real interview use: HTTP request builders, SQL query builders, config objects, immutable DTOs.

Don't confuse: Builder = assembling ONE complex thing across many steps. Factory = picking WHICH thing in one call.

4. Adapter (Structural)

Analogy: Foreign plug aur Indian socket. Dono sahi hain par fit nahi hote. Beech mein universal adapter laga do, ab dono jud jaate hain. Adapter kisi ko badalta nahi, bas dono interface ko match karwa deta hai.

What it does: Wraps one interface so it looks like the interface the client expects. Lets two incompatible classes work together.

Spot it when you hear:

- 'Integrate a third-party or legacy library'
- 'Two interfaces that don't match'
- 'Wrap an old API to fit new code'

Pseudocode

```
class XmlToJsonAdapter:  
    old = new XmlLibrary()  
    getJson():
```

```
xml = old.getXml()
return convert(xml)    # client only ever sees JSON
```

Real interview use: Payment gateway wrappers, legacy API integration, third-party SDK glue code.

Don't confuse: Adapter = make two INCOMPATIBLE interfaces fit. Facade = put ONE simple face over a complex subsystem.

5. Decorator (Structural)

Analogy: Pizza pe layers chadhaao. Base pizza lo, upar cheese, phir olives, phir paneer. Har layer pizza ko wrap karke kuch add karti hai aur price bhi badhati hai. Base waisa ka waisa rehta hai, bas upar add hota jaata hai.

What it does: Wraps an object to add new behavior at runtime, without changing the original class or any other object.

Spot it when you hear:

- 'Add features dynamically, one on top of another'
- 'Many optional add-ons in any combination'
- 'Avoid a subclass explosion for every combo'

Pseudocode

```
coffee = new SimpleCoffee()
coffee = new MilkDecorator(coffee)    # wraps, adds milk + cost
coffee = new SugarDecorator(coffee)  # wraps again, adds sugar
coffee.cost()    # each wrapper adds to the inner object's cost
```

Real interview use: Coffee / pizza pricing, I/O streams (BufferedReader wraps Reader), UI styling.

Worth knowing: Order of wrapping matters. BufferedReader(FileReader) is right, the reverse is not. Each wrapper adds one specific thing, so stack them in the order you need.

Don't confuse: Decorator = wrap to ADD new behavior. Proxy = wrap to CONTROL access (behavior stays the same).

6. Facade (Structural)

Analogy: Restaurant ka waiter. Tum sirf 'ek thali' bolte ho. Andar kitchen, chef, store-room ka

poora jhanjhat waiter sambhalta hai. Tumhe ek simple window milti hai, peeche ka complex system chhupa rehta hai.

What it does: Gives one simple entry point that hides a complex subsystem of many classes behind it.

Spot it when you hear:

- 'Too many classes to call in the right order'
- 'Simplify a messy subsystem for callers'
- 'One clean API over many internal steps'

Pseudocode

```
class OrderFacade:
    placeOrder(cart):
        payment.charge(cart)
        inventory.reserve(cart)
        shipping.schedule(cart)
    # caller calls ONE method, not three subsystems
```

Real interview use: Checkout flow, compiler front-end, an SDK that hides many internal services.

Don't confuse: Facade = SIMPLIFY one complex subsystem. Adapter = MATCH two incompatible interfaces.

7. Proxy (Structural)

Analogy: Club ka bouncer. Andar jaane se pehle wo ID check karta hai, toh tum seedhe andar nahi ghus sakte. Wahi bouncer kabhi tumhe gate pe rok ke rakhta hai jab tak andar ready na ho (lazy load), ya tumhari pichhli entry yaad rakh leta hai (cache). Asli club waisa ka waisa, bas beech mein ek gatekeeper.

What it does: Puts a stand-in in front of the real object to control access, lazy-load, cache, or log, without changing the real object.

Spot it when you hear:

- 'Control access or permissions to an object'
- 'Lazy-load an expensive object only when needed'
- 'Add caching or logging around calls'

Pseudocode

```
class ImageProxy:
```

```

real = null
display():
    if real is null:
        real = new HeavyImage()    # load only when needed
    real.display()                  # control around the real call

```

Real interview use: Lazy-loaded images, access control, caching proxy, rate limiting.

Don't confuse: Proxy = same interface, CONTROL access. Decorator = same interface, ADD behavior.

8. Strategy (Behavioral)

Analogy: Google Maps route options. Manzil same hai, par tum car, bike ya paidal raasta chun sakte ho. Algorithm yaani raasta tum bahar se switch karte ho, baaki sab waisa rehta hai.

What it does: Defines a family of interchangeable algorithms and lets you swap one for another at runtime, from outside.

Spot it when you hear:

- 'Multiple ways to do the same thing'
- 'Switch the algorithm at runtime'
- 'An if-else that just picks which method to run'

Pseudocode

```

class Sorter:
    strategy          # set from outside
    sort(data):
        return strategy.run(data)    # behavior chosen externally
# sorter.strategy = new QuickSort() OR new MergeSort()

```

Real interview use: Payment methods, sorting / compression algorithms, pricing rules, route planners.

Worth knowing: Why composition, not inheritance? A new algorithm is just a new strategy object. The Sorter class never changes. Subclassing would force a new subclass for every algorithm.

Don't confuse: Strategy = YOU pick the algorithm from outside. State = the OBJECT changes its own behavior as its status changes.

9. Observer (Behavioral)

Analogy: Instagram follow. Tum kisi ko follow karte ho. Jab wo post daalta hai, notification automatically saare followers ko chala jaata hai. Followers ko baar-baar check nahi karna padta, khabar khud aati hai.

What it does: When one object (the subject) changes, all its registered watchers (observers) get notified automatically.

Spot it when you hear:

- 'Notify many objects when one thing changes'
- 'Publish-subscribe / event listeners'
- 'Keep several views in sync with one model'

Pseudocode

```
class Channel:
    subscribers = []
    publish(post):
        for s in subscribers:
            s.notify(post)    # one change, fan-out to all
```

Real interview use: Notification systems, UI event listeners, stock / score tickers, pub-sub.

Worth knowing: Modern systems run on this everywhere: event buses, listeners, and pub-sub architectures are all Observer at scale. The Insta example is just the smallest version.

Don't confuse: Observer = one subject, many auto-notified watchers. Mediator = many objects talk through one central hub.

10. State (Behavioral)

Analogy: Traffic signal. Red pe ruko, green pe chalo, yellow pe slow. Same signal, par uska behavior current state pe depend karta hai. State badalte hi behavior khud badal jaata hai, tumhe har jagah if-else nahi likhna padta.

What it does: Lets an object change its behavior when its internal state changes, as if it switched class, instead of a giant if-else on a status field.

Spot it when you hear:

- 'Behavior depends on the current status'
- 'Lots of if status == PLACED / SHIPPED / ...'

- 'A state machine or workflow lifecycle'

Pseudocode

```
class Order:
    state          # Placed / Shipped / Delivered
    next():
        state = state.next()  # each state knows its own transition
```

Real interview use: Order / workflow lifecycle, vending machine, game character states, TCP connection.

Don't confuse: State = object changes its OWN behavior as status changes. Strategy = caller swaps the algorithm from OUTSIDE.

11. Command (Behavioral)

Analogy: Restaurant ka order slip. Tumhari request ek parchi yaani object ban jaati hai. Wo parchi kitchen ko jaati hai, baad mein execute hoti hai, aur galti ho toh cancel yaani undo bhi ho sakti hai.

What it does: Turns a request into a standalone object, so you can queue it, log it, pass it around, and undo or redo it.

Spot it when you hear:

- 'Support undo / redo'
- 'Queue or schedule requests'
- 'Decouple the thing asking from the thing doing'

Pseudocode

```
class PasteCommand:
    execute(): doc.paste()
    undo():    doc.removeLastPaste()
history.push(cmd); cmd.execute()  # later: history.pop().undo()
```

Real interview use: Undo / redo, task queues, transaction logs, remote-control buttons, menu actions.

Don't confuse: Command = wrap a REQUEST as an object (for undo / queue). Strategy = wrap an ALGORITHM choice.

12. Template Method (Behavioral)

Analogy: Maggi recipe. Steps fixed hain: paani garam karo, maggi daalo, masala daalo, pakao. Par 'masala' wala step tum apne hisaab se badal sakte ho. Skeleton same rehta hai, sirf ek-do steps tum override karte ho.

What it does: Defines the skeleton of an algorithm in a base class and lets subclasses override specific steps, without changing the overall structure.

Spot it when you hear:

- 'A fixed sequence where some steps vary'
- 'Same workflow, different details per type'
- 'Avoid duplicating the same algorithm structure'

Pseudocode

```
class Report:
    generate():          # fixed skeleton, do NOT override
        fetch()
        format()        # subclass overrides only this step
        export()
# PdfReport overrides format(); CsvReport overrides format()
```

Real interview use: Data parsers, report generators, framework hooks, game turn loops.

Don't confuse: Template Method = base class FIXES the steps, subclass fills blanks (inheritance). Strategy = whole algorithm swapped via composition.

8. The confusion-killer - the 5 pairs people always mix up

This table exists because these five pairs are where students lose the interview. Memorize the tell in the right column.

The 5 pairs people always mix up

This is where the interview is won or lost. Memorize the tell.

Factory One call: 'give me the right type'.	vs	Builder Many steps: 'assemble one complex object'.
Strategy YOU pick the algorithm from outside.	vs	State The OBJECT switches its own behavior as status changes.
Decorator Wraps to ADD new behavior.	vs	Proxy Wraps to CONTROL access (behavior unchanged).
Adapter Makes two incompatible interfaces fit.	vs	Facade One simple face over a complex subsystem.
Observer One subject, many auto-notified watchers.	vs	Mediator Many objects talk through one central hub.

The 5 mix-ups. Left vs right - the tell is all you need to remember.

Often confused	The one tell that separates them
Factory vs Builder	Factory = one call, 'which type?'. Builder = many steps, 'assemble one complex object'.
Strategy vs State	Strategy = YOU pick the algorithm from outside. State = the OBJECT switches its own behavior as status changes.
Decorator vs Proxy	Both wrap an object. Decorator ADDS new behavior. Proxy CONTROLS access (behavior unchanged).
Adapter vs Facade	Adapter = make two incompatible interfaces fit. Facade = one simple face over a complex subsystem.
Observer vs Mediator	Observer = one subject, many auto-notified watchers. Mediator = many objects talk through one central hub.

9. Machine-coding rounds - which pattern shows up where

Machine-coding rounds give you a small system and 60-90 minutes to design and code it. You are not expected to cram in patterns. You ARE expected to spot the one or two spots where a pattern makes the design cleaner. Here is the cheat-map for the classic problems.

Problem	Patterns that usually fit	Why
Parking Lot	Factory (spot / vehicle), Strategy (pricing), Singleton (the lot)	The classic warm-up. Vehicle types vary (Factory), pricing rules vary (Strategy).
Elevator system	State (idle / moving / door), Strategy (scheduling), Observer (requests)	Heavily a state machine. The lift's behavior depends on its current state.

Problem	Patterns that usually fit	Why
Movie booking (BookMyShow)	Singleton (seat-lock manager), Observer (seat availability), Strategy (payment)	Concurrency plus notifying users when seats free up.
Vending machine	State (idle / has-money / dispensing), Strategy (payment)	The textbook State example. Each state allows different actions.
Splitwise	Strategy (equal / exact / percent split), Observer (balance updates)	Split logic varies, so it is a Strategy. Balances update many views.
Logging framework	Singleton (one logger), Chain of Responsibility (log levels), Strategy (output sink)	One shared logger, levels passed down a chain until one handles it.

How to attack ANY machine-coding problem (4 steps)

- Find the nouns (entities). These become your classes. Parking lot, spot, vehicle, ticket.
- Find the verbs (actions). These become your methods. park, unpark, calculateFee.
- Find what VARIES. Variation is where a pattern fits: varying algorithm -> Strategy, varying status -> State, varying creation -> Factory.
- Start simple. Add a pattern ONLY when the interviewer pushes for flexibility ('what if pricing changes?'). Never force a pattern to show off.

10. Interview triggers - if they say X, reach for Y

If the interviewer says...	...reach for
'There should be only one of these'	Singleton
'Create objects without naming the exact class'	Factory
'This constructor has too many parameters'	Builder
'Make these two incompatible things work together'	Adapter
'Add features without changing the class'	Decorator
'Simplify this complex subsystem for callers'	Facade
'Control access, lazy-load, or cache this'	Proxy
'Switch the algorithm at runtime'	Strategy
'Notify many objects when one changes'	Observer
'Behavior depends on the current status'	State
'Support undo and redo'	Command
'Same steps, different details each time'	Template Method

From problem statement to pattern - 3 worked examples

The hard part is not naming a pattern. It is spotting it inside the interviewer's words. Here is the leap, three times.

- 'Design a logger that can switch its output to file, console, or cloud at runtime.' -> the

behavior is chosen from outside -> Strategy. A SinkStrategy interface, the logger holds one and calls strategy.write().

2. 'An order moves through placed -> shipped -> delivered, and each stage allows different actions.' -> behavior depends on the current status -> State. Each state object knows its own allowed actions and the next state.

3. 'Add gift-wrap and express-shipping as optional add-ons to a product, in any combination, each adding cost.' -> wrap to add behavior -> Decorator. Each add-on wraps the product and adds to cost().

11. Common mistakes in LLD rounds

Mistake	Why it loses points	Do this instead
Forcing patterns everywhere	Over-engineering. Stacking Singleton + Factory + Strategy on a 2-class problem screams 'I memorized patterns'.	Start with plain classes. Add a pattern only when a real need (changing requirement) appears.
Making everything a Singleton	Global state, hard to test, hidden coupling. Singleton is the most over-used pattern.	Use it only for genuinely single things (config, logger). Prefer passing dependencies in.
One god class doing everything	Violates Single Responsibility. Becomes unmaintainable and impossible to explain.	Split by responsibility. One class, one job.
Confusing Strategy and State	The single most common mix-up. Interviewer notices instantly.	Strategy = you swap the algorithm from outside. State = object swaps its own behavior as status changes.
Coding to concrete classes	Violates Dependency Inversion. Hard to extend or mock in tests.	Depend on interfaces. 'new ConcreteThing' should be rare and isolated.
Memorizing names, not the WHEN	You recite the definition but blank on 'why this pattern here?'.	Learn the trigger (section 10) and the analogy. The name is the last thing that matters.

12. The other 11 GoF patterns - recognize the words

There are 23 GoF patterns in total. The 12 above cover almost every fresher and mid-level interview. The remaining 11 are good to recognize. Each gets a one-line hook too, so the name rings a bell. Know the hook, do not stress the implementation.

Pattern	Type	One line to remember	What it does
Abstract Factory	Creational	Factory ka bhi factory. (poora UI kit ek saath)	Creates whole families of related objects together (all Windows widgets, or all Mac widgets).
Prototype	Creational	Naya banane ki jagah copy maar lo. (Ctrl+C Ctrl+V)	Clone an existing object instead of building a new one from scratch.
Bridge	Structural	Remote alag, TV alag, dono apne hisaab se badlo.	Split an abstraction from its implementation so both can vary independently.
Composite	Structural	Folder ke andar folder, sab same tarah treat. (file tree)	Treat a tree of objects and a single object the same way (folders and files).
Flyweight	Structural	Same cheez baar-baar mat banao, share karo. (chess pawns)	Share common data across many objects to save memory.
Chain of Responsibility	Behavioral	Request aage badhti jaaye jab tak koi pakde. (support L1-L2-L3)	Pass a request along a chain until someone handles it.
Iterator	Behavioral	Ek-ek karke ghoomo, andar ka structure mat poochho. (for-each)	Walk through a collection without exposing how it is stored.
Mediator	Behavioral	Sab ek hub ke through baat karein. (air traffic control)	Objects talk through one central hub instead of directly with each other.
Memento	Behavioral	State ka snapshot le lo, baad mein wapas. (save game)	Save and restore an object's state without exposing its internals.
Visitor	Behavioral	Naya kaam baahar se add karo, class ko chhuo mat.	Add new operations to objects without changing their classes.
Interpreter	Behavioral	Chhoti si bhasha banao aur padho. (calculator expression)	Define a small grammar and evaluate sentences written in it.

If you have the core 12 solid, these 11 are pure bonus. In a fresher round you will almost never be asked to code one. Recognize the name, say the one-liner, move on.

13. Revision - the last 20 minutes before the round

The 12 analogies, one line each

- Singleton - ek hi Principal, har baar wahi banda.
- Factory - chai tapri, tum maango, kitchen decide kare kaunsi banegi.
- Builder - Subway sandwich, step by step, phir .build().
- Adapter - foreign plug + Indian socket, beech mein adapter.
- Decorator - pizza pe layers, base same, upar add karte jao.
- Facade - waiter, tum order do, kitchen ka jhanjhat wo sambhale.

- Proxy - club ka bouncer, andar jaane se pehle ID check.
- Strategy - Google Maps, manzil same, raasta tum chuno.
- Observer - Insta follow, post daalo, sab followers ko notification.
- State - traffic signal, behavior current state pe depend.
- Command - order slip, request ek object, baad mein execute / undo.
- Template Method - Maggi recipe fixed, sirf masala step override karo.

The 5 you must be able to code

Singleton, Factory, Strategy, Observer, Decorator. If you can write these five from memory, you clear most fresher LLD rounds.

The 3 confusion tells to re-read right before going in

- Strategy = you pick the algorithm. State = object switches its own behavior.
- Decorator = adds behavior. Proxy = controls access.
- Adapter = fits two interfaces. Facade = simplifies one subsystem.

You learned 12 patterns as 12 pictures, not 12 definitions. The interviewer will describe a situation, not name a pattern. Match the situation to a picture, say the name, then code the core 5 you practised. You have done this. Go.